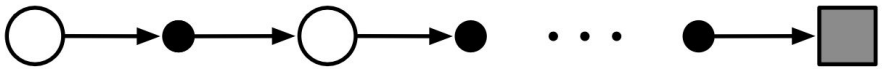
Temporal-Difference (TD) Learning

Genya Ishigaki

Revisiting MC Prediction

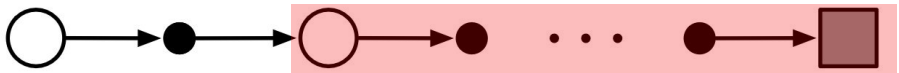
- MC Prediction updates the action value function

$$V(S_t) \leftarrow V(S_t) + \alpha\left[G_t - V(S_t)\right]$$



- Remember the Bellman Expectation Equation
  - A sampled return can be replaced with a sampled reward and the expected action value from the successor state with the next action

$$R_{t+1} + \gamma V(S_{t+1})$$



TD Prediction: TD(0)

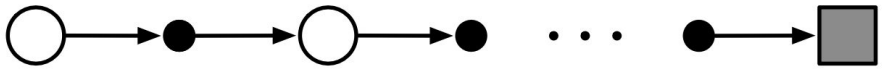
Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

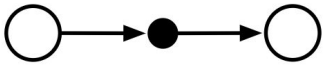
Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

MC vs. TD(0)

- Target value
  - MC: A sampled return G
  - TD(0): A sampled reward and the action value R + Q(S’, A’)
- Update
  - MC: Wait until the return following the visit is known
  - TD(0): Wait only until the next time step (bootstrapping)
    - At time t + 1 they immediately form a target and make a useful update using the observed reward



MC



TD(0)

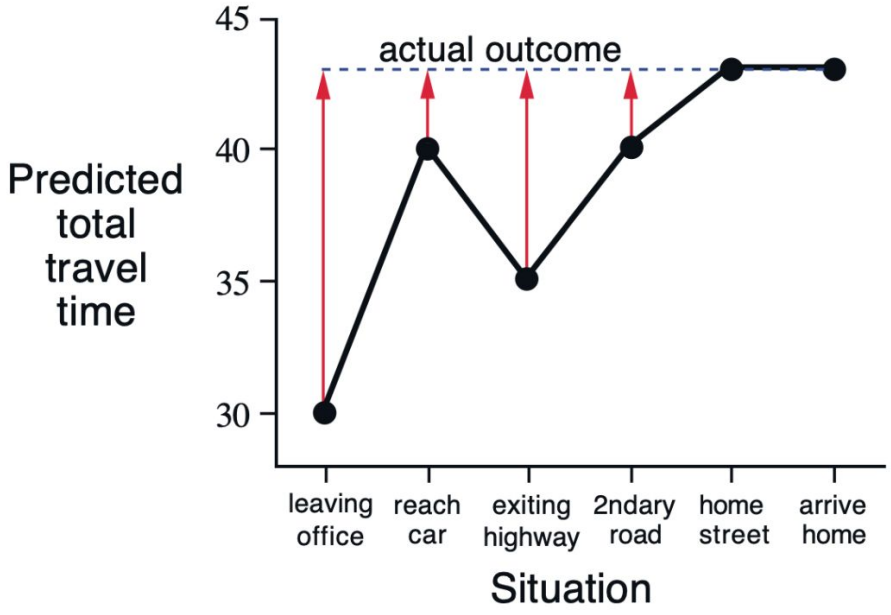
An Example: Driving Home

State	*Elapsed Time (minutes)*	*Predicted Time to Go*	*Predicted Total Time*
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

- The rewards in this example are the elapsed times on each leg of the journey
- No discounting
- The value of each state is the expected time to go

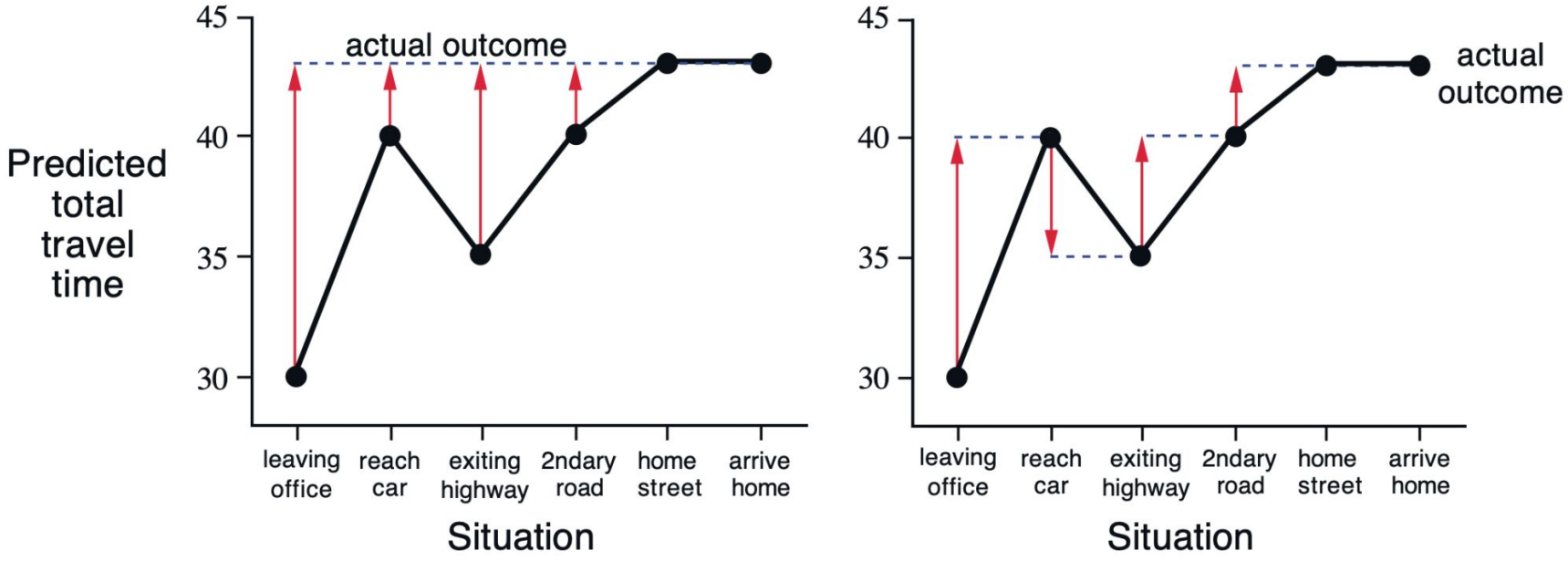
An Example: Driving Home

- MC suggests updates based on the actual outcome after the entire travel process is done



An Example: Driving Home

- TD(0) suggests updates at each time step (even before the actual travel time gets known)



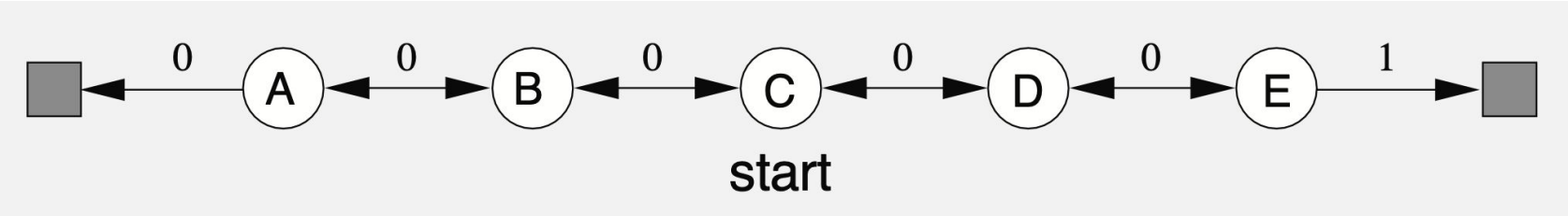
An Example: Driving Home

- Is it reasonable to suggest updates even on the way?
  - You become stuck in a massive traffic jam
  - Twenty-five minutes after leaving the office, you are still bumper-to-bumper on the highway
- As you wait in traffic, you already know that your initial estimate of 30 minutes was too optimistic
  - MC: Still wait until the end to update the state value (expected waiting time)
  - TD(0): Learn immediately, shifting your initial estimate

Advantages of TD(0)

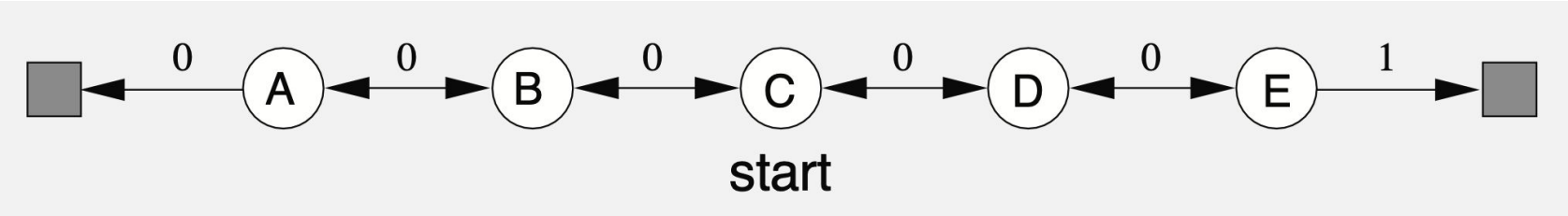
- Naturally implemented in an online, fully incremental fashion
  - Some applications have very long episodes, so that delaying all learning until the end of the episode is too slow
  - Other applications are continuing tasks and have no episodes at all (MC cannot be used for these cases)
- TD has low variance, some bias
- MC has high variance, zero bias

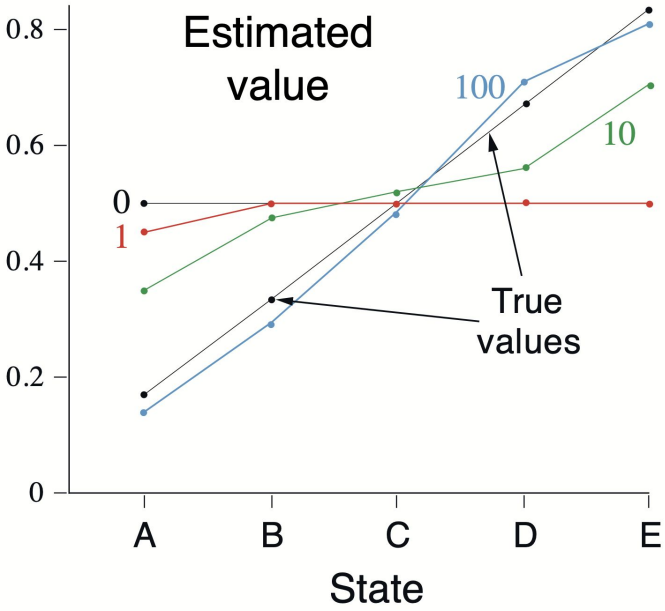
Comparing MC and TD(0): Random Walk



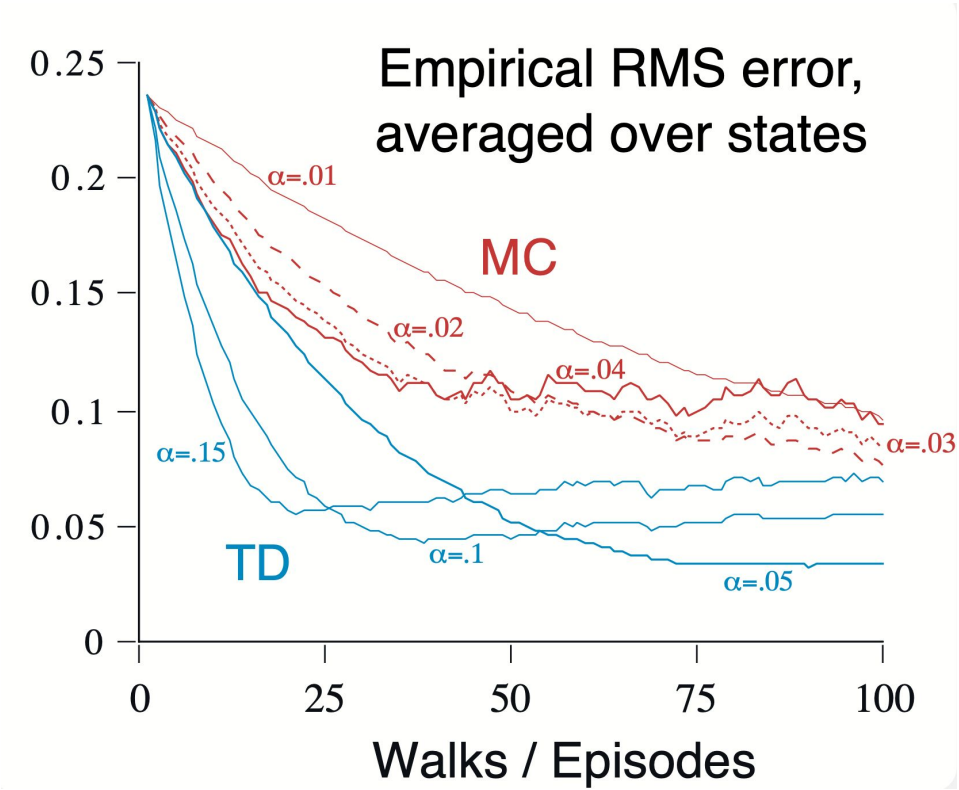
- Can you guess which state(s) has a higher value under a random walk? (with some discount)

Comparing MC and TD(0): Random Walk





Comparing MC and TD(0): Random Walk



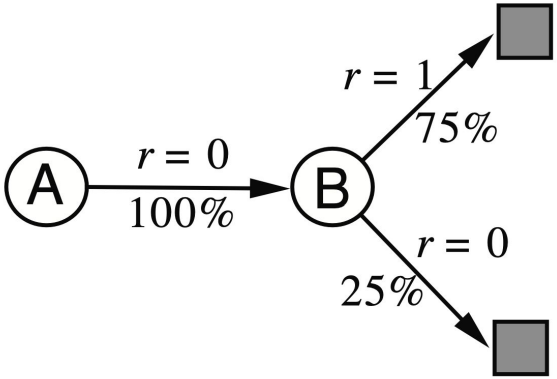
* Root Mean Square (RMS) Error

Milestone Questions: Batch Updates

- Assume:
  - Two states A, B
  - No discounting
  - The following eight episodes will be sampled again and again
    - A 0 B 0
    - B 1
    - B 1
    - B 1
    - B 1
    - B 1
    - B 1
    - B 0
- Compute V(A) and V(B) with MC or TD(0)

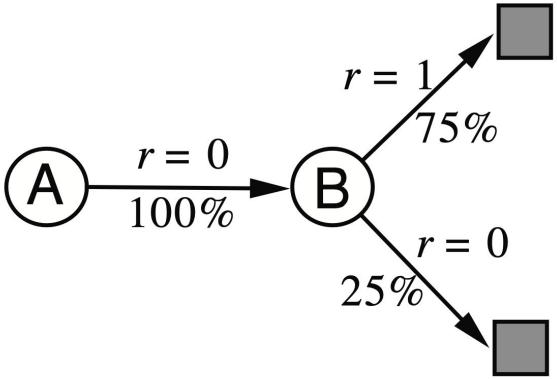
Milestone Questions: Batch Updates

- MC
  - V(A) = 0
  - V(B) = 6 / 8
- TD(0)



Milestone Questions: Batch Updates

- MC
  - V(A) = 0
  - V(B) = 6 / 8
- TD(0)
  - V(A) = 6 / 8 (Bootstrapping)
  - V(B) = 6 / 8



Sarsa: On-policy TD Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Milestone Questions

- Why is Sarsa on-policy? You can assume the epsilon-greedy algorithm.

Q-learning: Off-policy TD Control

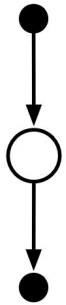
Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

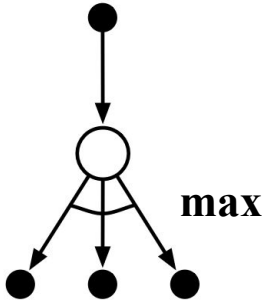
Sarsa vs. Q-learning

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]$$



Sarsa

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$



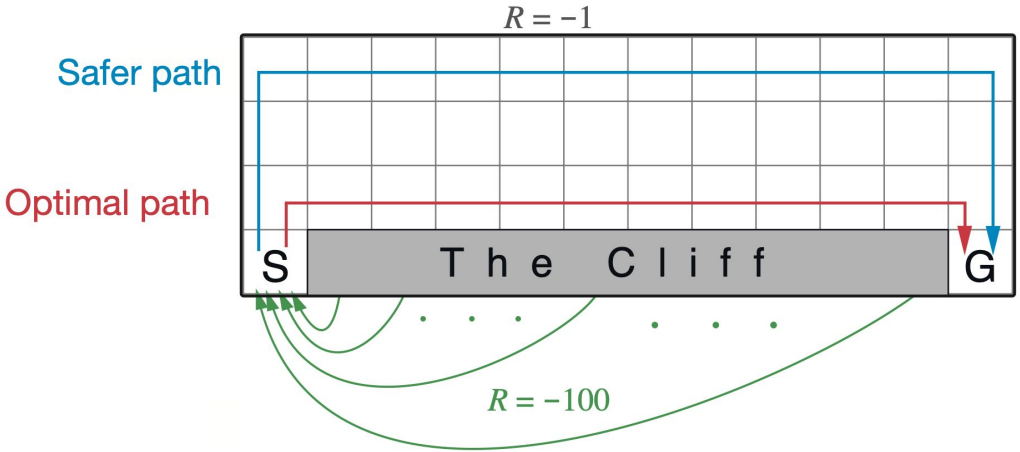
Q-learning

Milestone Questions

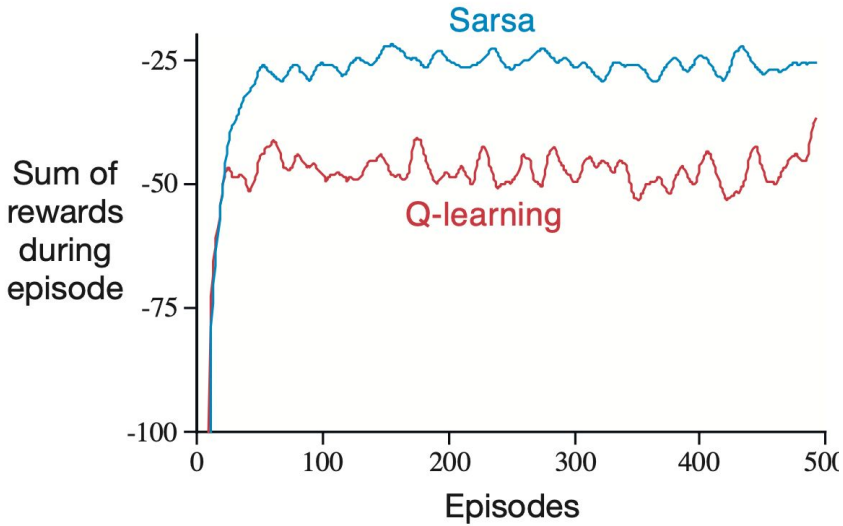
- There are two actions (Left or Right) from each state
- The current Q table shows
  - Q(S1, L) = 1.0, Q(S1, R) = 0
  - Q(S2, L) = - 1.0, Q(S2, R) = 2.0
  - Q(S3, L) = 5.0, Q(S3, R) = - 1.0
- Assume an agent observed the following experiences
  - S1 L 3 S2
  - S2 R 4 S3
- This episode was generated, following the epsilon-greedy
- Using Q-learning,
  - Compute the action value (assume no discounting for simplicity)
  - Update the policy function (alpha = 0.1 for simplicity)

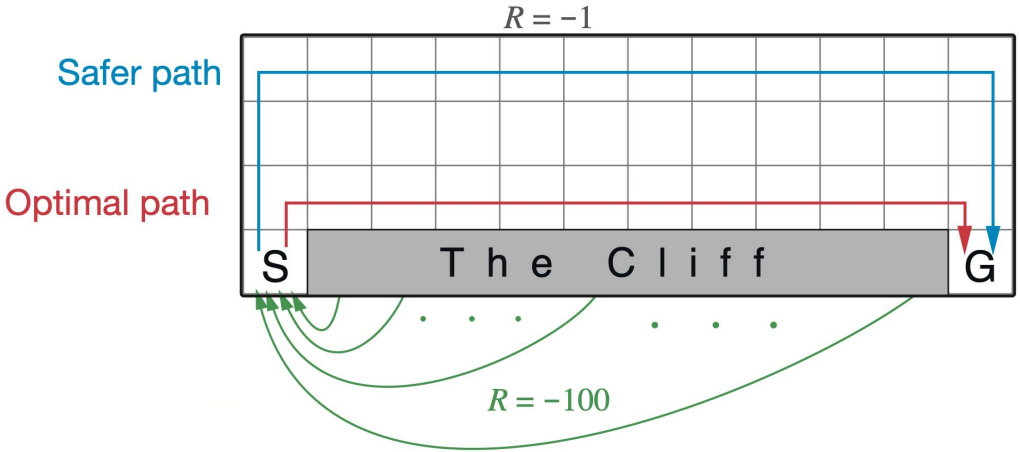
An Example: Cliff Walking

- Reward
  - -1 on all transitions except those into the region marked "The Cliff"
  - -100 Stepping into the cliff
- Termination
  - Reaching the goal state
  - Stepping into the cliff
- No discounting



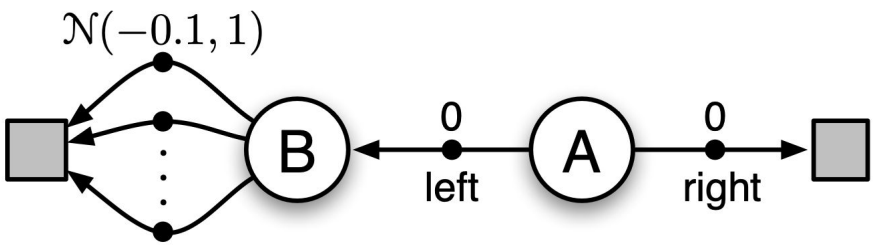
An Example: Cliff Walking



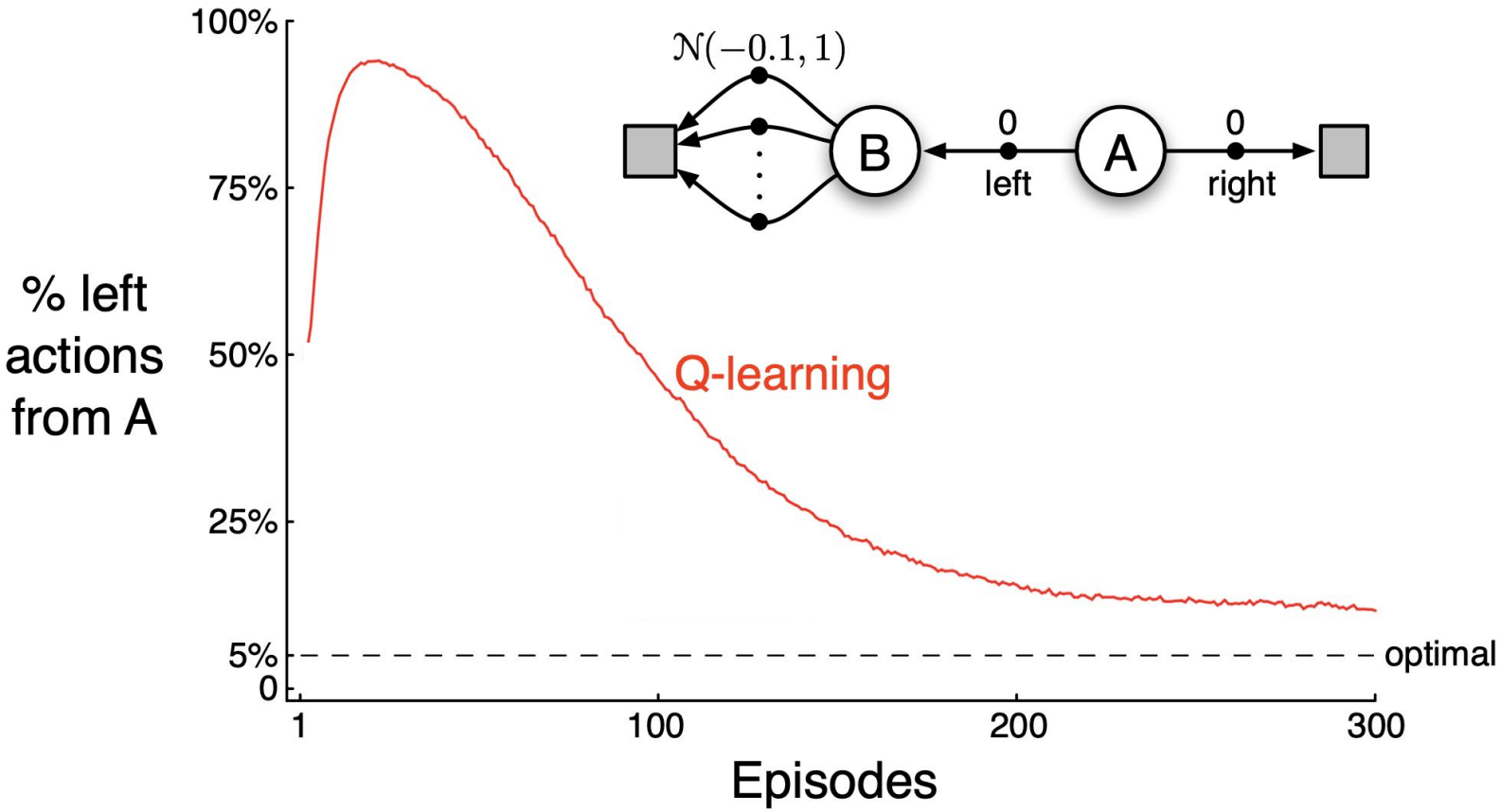


The Maximization Bias

- Q-learning with a max operation
- These maximization in their policy gives positive bias to the estimation of the value functions
- Assume the following toy example
  - N(-0.1, 1): A normal distribution with a mean of -0.1 and variance of 1
  - Overall, it's better to go right from State A



The Maximization Bias



Mitigating the Maximization Bias

- Using the same samples (plays) both to determine the maximizing action and to estimate its value
- We divided the plays in two sets and used them to learn two independent estimates: $Q_1(s, a)$ and $Q_2(s, a)$
- Coin-flip after each experience:
  - If the coin comes up heads, the update is on Q_1
  - Otherwise, on Q_2
- This concept is called *double learning*

Double Q-learning

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

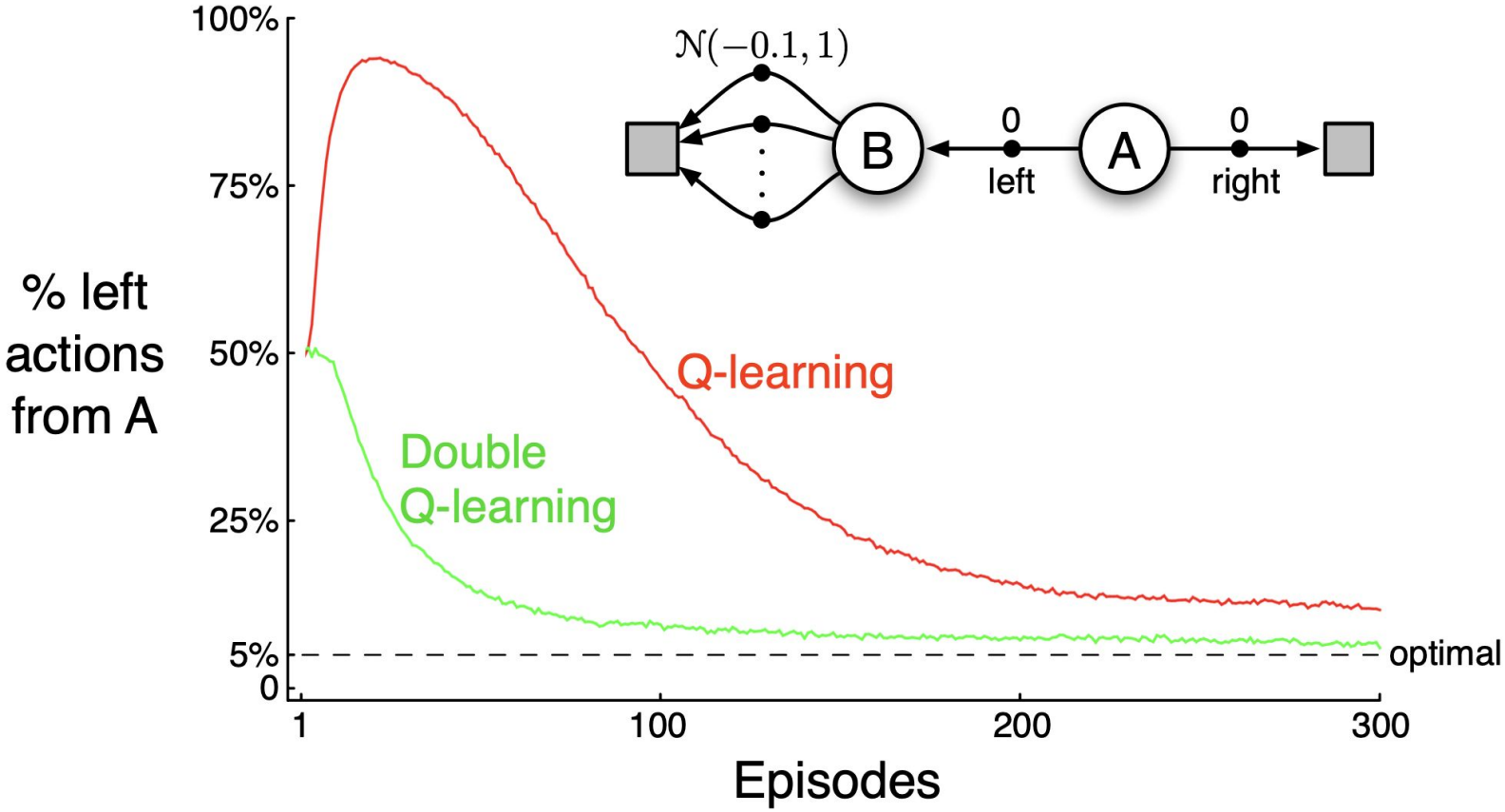
$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

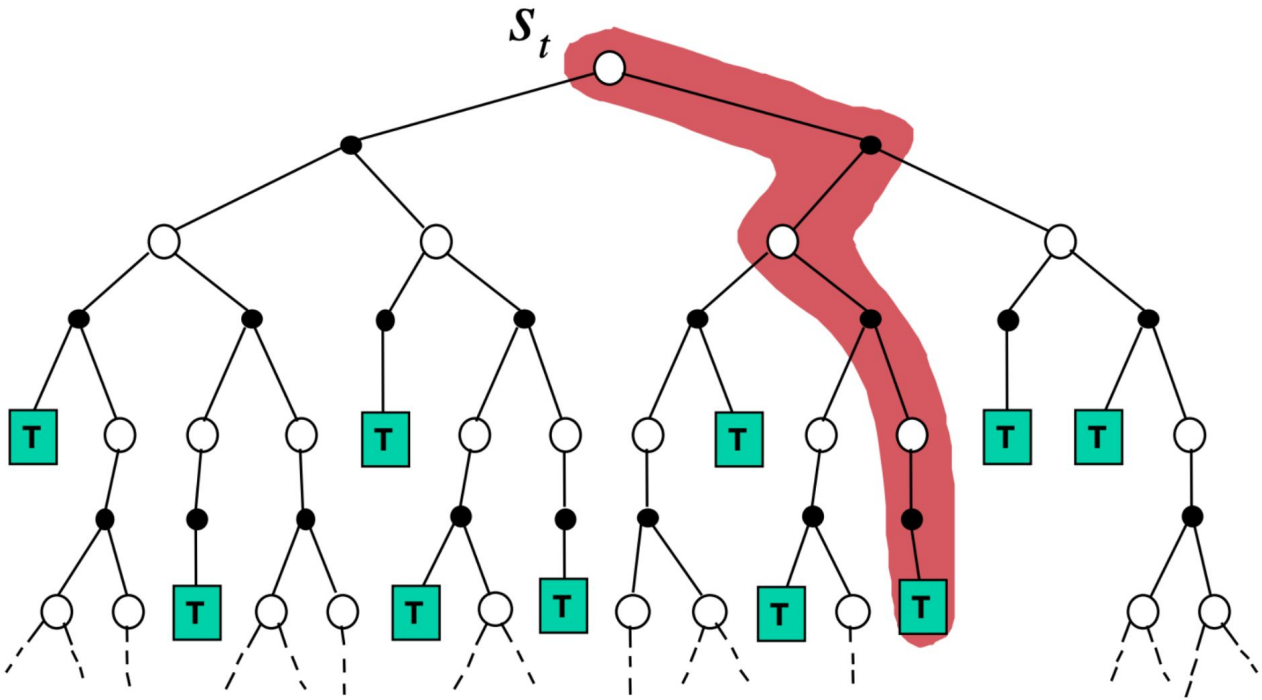
    $S \leftarrow S'$
  until S is terminal

The Maximization Bias



Monte Carlo Methods

$$V(S_t) \leftarrow V(S_t) + \alpha\left(G_t - V(S_t)\right)$$

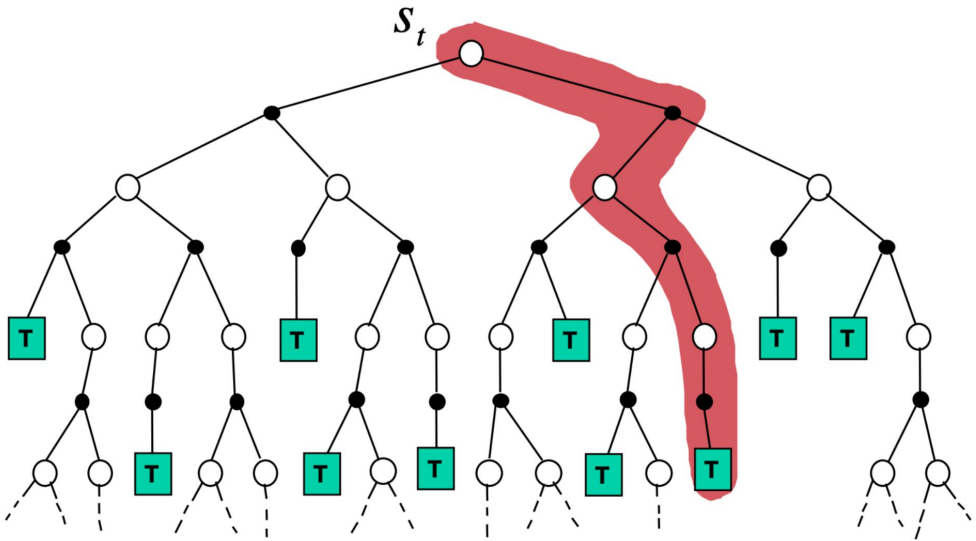


- Episode sampling
- No bias
- High variance

Milestone Question

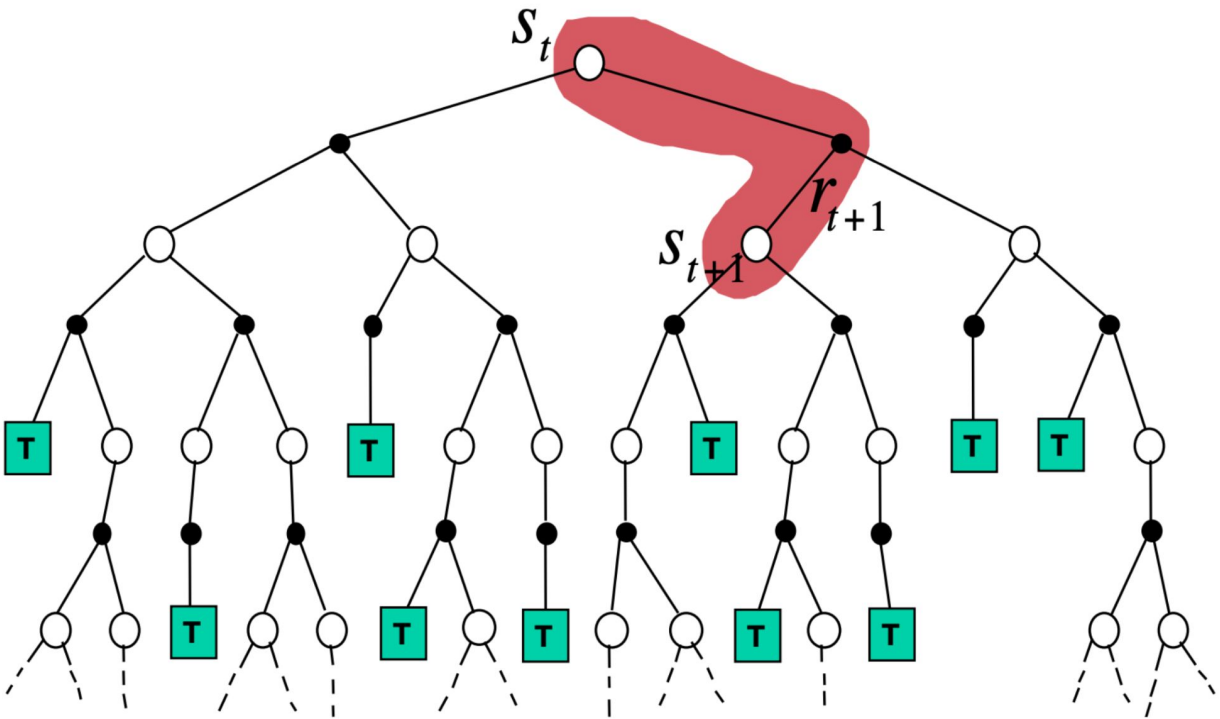
- Draw a similar diagram for TD and DP (based on the MC diagram)

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



Temporal-Difference Learning

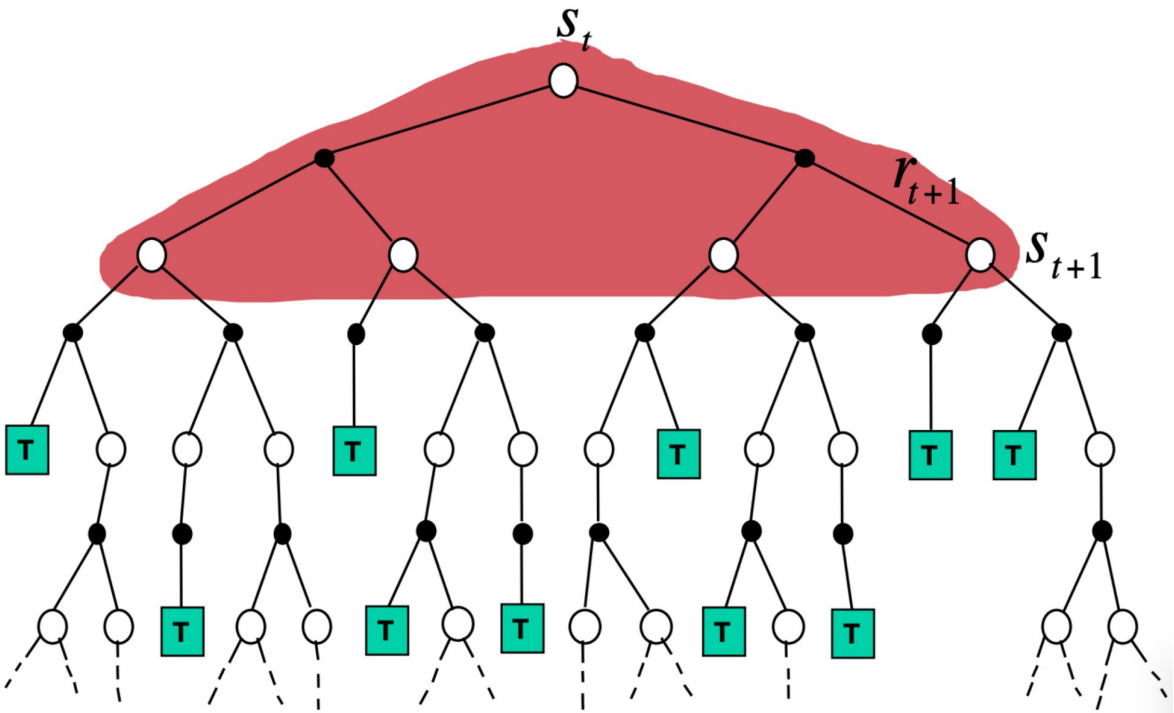
$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right)$$



- Bootstrapping
- Reward sampling
- Some bias
- Less variance

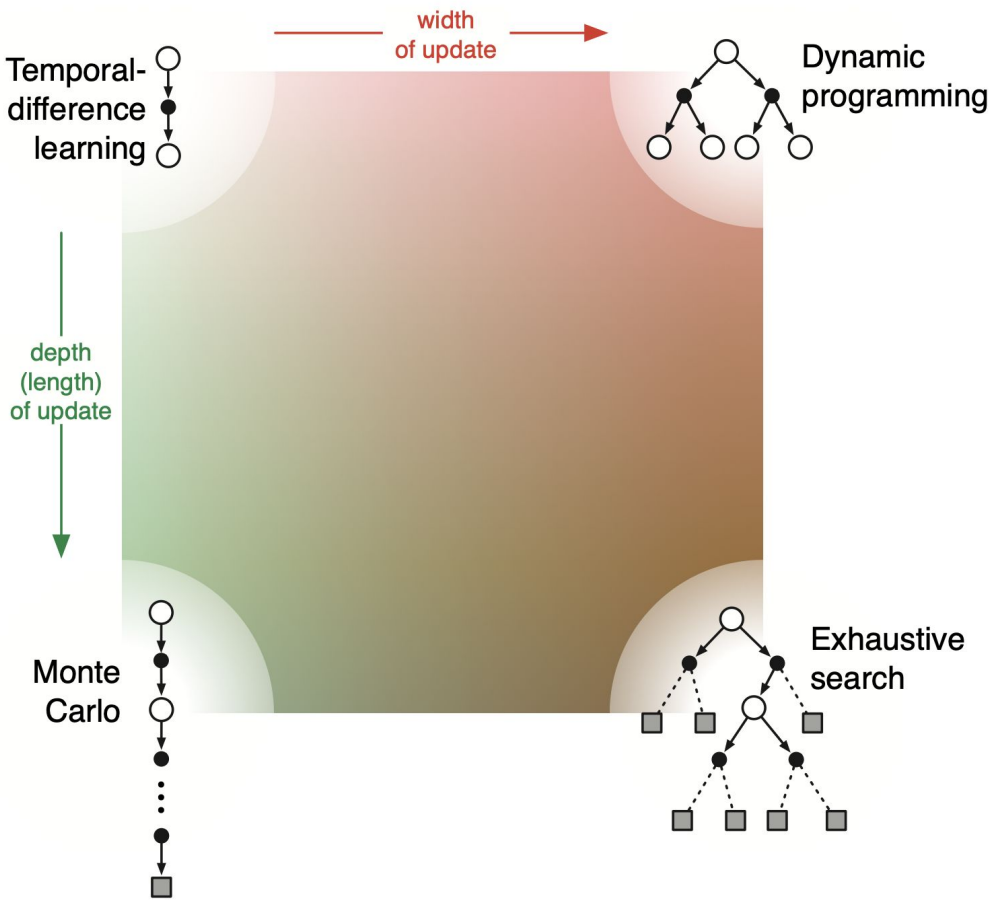
Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_\pi \left[R_{t+1} + \gamma V(S_{t+1}) \right]$$



- Bootstrapping
- Exhaustive estimation

Summary


width
of update
Temporal-
difference
learning
Dynamic
programming
depth
(length)
of update
Monte
Carlo
Exhaustive
search